



Kubernetes CRD & Operator



What is a CRD (Custom Resource Definition)?

- CRD (CustomResourceDefinition) allows you to extend the Kubernetes API
- by creating your own custom resource types.
- 👉 Just like:
 - Pods
 - Services
 - Deployments
- 🎯 CRD = Create your own Kubernetes resource types



Why CRD?

- Kubernetes built-in default resources are not always enough.
 - Pod
 - Service
 - Deployment
- 👉 But real-world needs require:
 - Database management 🗄️
 - Traffic routing 🌐
 - Certificates 🔒
 - GitOps applications
- CRDs solve this by allowing custom objects .



CRD = Extending Kubernetes API

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
```

- Adds new resource types to Kubernetes



CRD vs CR

 Term	 Meaning	 How It Works	 Example
 CRD (CustomResourceDefinition)	 Defines new resource type	 Extends Kubernetes API with custom objects	Define MyApp , Database , Backup
 CR (Custom Resource)	 Instance of that type	 Actual object created using that CRD	myapp-prod , db-instance-1

Example

 Argo CD:

- CRD → Application
- CR → my-app (actual deployment)

How CRDs Work

Apply CRD 
↓
Kubernetes learns new type 
↓
Create Custom Resource (CR) 
↓
Kubernetes manages it like native resource 

Real-World CRDs

 CRD	 Tool	 Purpose	 How It Works	 Real-World Insight
 Application	 Argo CD	GitOps deployment	 Defines desired state from Git → auto sync to cluster	 Automates deployments
 VirtualService	 Istio	Traffic routing	 Controls routing (paths, headers, canary releases)	 Advanced traffic control
 Certificate	 Cert-Manager	TLS management	 Issues & renews certificates automatically	 HTTPS automation
 SealedSecret	 Bitnami	Encrypted secrets	 Encrypts secrets → safe to	 Secure GitOps

 CRD	 Tool	 Purpose	 How It Works	 Real-World Insight
			store in Git	workflows

CRD Details

- apiVersion: apiextensions.k8s.io/v1
- Stored in etcd Just like: Pods , Services
- Managed like normal Kubernetes objects

Useful Commands

```
kubectl get crds          #  List CRDs
kubectl get <resource-name> #  List Custom Resources
```

What Happens If You Delete a CRD?

-  Important behavior:
 -  All custom resources are deleted
 -  Kubernetes stops recognizing that resource

What is an Operator?

- An Operator automates application management using:
 - CRDs
 - Controllers
- Operator = CRD + Controller (automation engine)

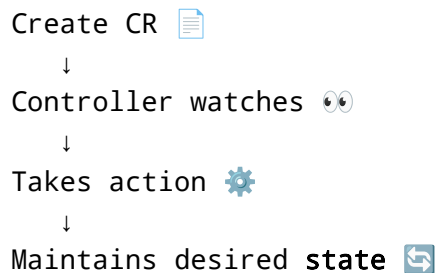
Analogy

- Think of Operator as:
 -  Kubernetes = OS
 -  Operator = Smart automation robot
 - Handles complex tasks automatically

Key Components

 Component	 Role	 How It Works	 Real-World Example
 CRD (CustomResourceDefinition)	Defines resource	 Extends Kubernetes API with new resource type	MyApp , Database , Backup
 CR (Custom Resource)	Actual object	 Instance created from CRD	myapp-prod , db-prod
 Controller	Watches & acts	 Continuously monitors CR and takes action to match desired state	Creates Pods, Services automatically
 Operator	Full automation system	 Combines CRDs + Controllers + domain logic	Automates DB setup, scaling, backup

How Operator Works



How Operator Works

 Step	 Action	 What Happens Internally	 Why It Matters
 1	 User creates CR	 User Defines desired state using Custom Resource (YAML)	 Declarative config
 2	 Controller watches	 Controller continuously watches for CR changes (watch API)	 Automation trigger
 3	 Operator acts	 Operator executes logic (create Pods, DB, backups, etc.)	 Handles complexity
 4	 Maintains state	 Reconciliation loop ensures actual = desired state	 Self-healing

 Operator	 Purpose	 What It Automates	 Real-World Use Case
 Argo CD Operator	GitOps automation	 Manages deployment lifecycle from Git (sync, rollback, self-heal)	 Continuous delivery using Argo CD
 Prometheus Operator	Monitoring setup	 Deploys Prometheus, Alertmanager, ServiceMonitors	 Observability stack
 Elasticsearch Operator	Logging stack	 Manages Elasticsearch clusters (scaling, recovery)	 Centralized logging (ELK)
 Database Operators	DB lifecycle	 Handles DB Provisioning, backups, scaling, failover	 MySQL/PostgreSQL automation

Benefits

-  Automates complex tasks
-  Extends Kubernetes capabilities
-  Reduces manual work
-  Improves reliability

Limitations

-  Requires learning CRDs & controllers
-  More complexity than basic YAML

Full Flow (CRD + Operator)

```

CRD  → Define type
↓
CR  → Create object
↓
Controller  → Watch changes
↓
Operator  → Take action
↓
System managed automatically 

```



Final Summary

- CRD = Extend Kubernetes API
- CR = Instance of CRD]
- Operator = Automates app lifecycle using CRDs + controllers
- Used in real-world tools like Argo CD , Istio
 - 🧩 Ultimate One-Line
 - CRDs extend Kubernetes, and Operators automate complex application management using those extensions.